Programming Task 2. Language Modeling, Fine-tuning

General Task: The main goal of this assignment is to learn how to implement custom text generation methods, understand different decoding strategies, and fine-tune pre-trained language models. You will implement a custom decoder with various sampling strategies (task 1) and fine-tune a GPT-2 model on instruction-following data (task 2). The two task can be solved independently, but it is recommended to complete task 1 first as it provides a foundation for understanding text generation and sampling strategies.

Dataset: For both tasks, you will be using the Alpaca instruction-following dataset, which contains instruction-input-response triples (as well as the formatted instructions) for training conversational AI models.
Hint: It is recommended to look through the data before starting the implementation.

Suggestions and hints:

- For both tasks, you are provided with function stubs in the `decoder.py` and `finetune.py` files. Each function stub contains a docstring that describes the expected input and output. The functions that you need to implement or complete are marked with the `TODO` comment.
- **Start early:** This assignment involves implementing non-trivial algorithms for text generation and model fine-tuning. Don't leave it until the last minute.
- **Understand decoding strategies:** Before writing any code, make sure you understand greedy decoding, temperature scaling, top-k sampling, and nucleus (top-p) sampling conceptually.
- **PyTorch fundamentals:** This assignment requires understanding `torch.Tensor` operations, and the basic training loop (forward pass, loss calculation, backward pass, optimizer step). Refer to the official PyTorch tutorials and documentation if needed.
- **Start small:** Begin with a small subset of the data to test your implementation (set `small=True` in the dataset loading functions). Once you have a working version, scale it up to the full dataset.
- **Debugging:** If you encounter issues, use print statements or a debugger to inspect the values of variables at different stages of your code. Use the provided logging functionality extensively. All methods should log their progress and results to help you debug issues.
- **GPU usage:** If available, use GPU acceleration for training. The code automatically detects and uses CUDA if available. There are also free online resources like Google Colab or Kaggle Notebooks that provide GPU access.
- **Consult resources (but implement yourself!):** Besides the suggested readings, you can look up explanations of the algorithms online. However, the goal is to implement it yourself, so avoid directly copying code from external sources. Understanding the algorithms will help you in the long run, especially for the exam.
- **Submission report:** Include a brief report summarizing your findings and analysis of the generated responses. The report should include qualitative and quantitative analysis of the generated responses, as well as a discussion of the strengths and weaknesses of the implemented methods. Use the assignment template [Overleaf] to write your report. Maximum length is **2 pages** (longer reports will not be accepted). The report should be submitted as a PDF file together with your code for grading.

Setup:

- Install dependencies: `pip install -r requirements.txt`
- Set your group number: Update the `GROUP` variable in `helper.py` with your group number.
- Test with small dataset first: Use `small=True` parameters for initial testing.
- Monitor training: Use the logging output to monitor training progress and debug issues.

Recommended Readings:

- Lecture slides. Text Models > Text Similarity
- Lecture slides. Language Models > Language Modeling
- Lecture slides. Language Models > Introduction to LLMs
- Lecture slides. Language Models > Text Generation
- Daniel Jurafsky, James H. Martin. 2025. Chapter 10: Large Language Models In Speech and Language Processing (3rd ed.).
- Radford, Alec, et al. Language models are unsupervised multitask learners." OpenAI (2019).

Task 1 (10 Points): Implement Custom Text Generation with Different Decoding Strategies (`decoder.py`). You need to implement the following methods:

(a) **Load the dataset:** Complete the `load_and_split_dataset` function in `helper.py`

(a1) Split the dataset into train and test sets.

(a2) Remove the gold standard responses from the test set to create a prompt-only test set.

(b) **Sampling and decoding strategies:** Implement various sampling strategies for token generation (`_sample_token`):

(b1) Greedy sampling: Select the token with the highest probability.

(b2) Temperature scaling: Apply temperature to logits before sampling.

(b3) Top-k filtering: Filter to top-k most likely tokens.

(b4) Top-p (nucleus) sampling: Filter to the smallest set of tokens whose cumulative probability exceeds top-p.

(b5) Ancestral sampling: Sample from the probability distribution.

(c) **Generate completions:** Use your implementation of the sampling strategies on a set of prompts (you can come up with your own examples) to generate responses. The function should take a prompt and return a generated response/completion based on the sampling strategy. You can use the pre-trained GPT-2 model and/or a fine-tuned GPT-2 (from task 2) for this.

(d) **Analysis:** Analyze the generated responses. Include the following in your submission:

(d1) A qualitative analysis of the generated responses, discussing the differences between the sampling strategies.

(d2) A quantitative analysis of the generated responses, such as perplexity or repetition score.

(d3) A brief discussion of the strengths and weaknesses of each sampling strategy based on the generated responses.

(e) **Submission:** Include the following in your submission:

- The analysis report summarizing the results of the different sampling strategies (one report for both tasks).
- Your implementation of the decoder in `decoder.py`.
- Generated outputs in `custom_decoder_results_GROUP_XX.txt`.
- Log file in `log_GROUP_XX.log`.
- A brief report summarizing your findings and analysis of the generated responses.

Task 2 (10 Points): Fine-tune GPT-2 Model (`finetuning.py`)

You need to implement the following methods in the `GPT2Model` class:

(a) **Tokenize the inputs:** Tokenize input examples with proper truncation and padding in the `tokenize_function`.

(b) **Training loop:** Complete the training loop (`train`) with:

- Forward pass through the model.
- Loss calculation.
- Backward pass and gradient updates.
- Learning rate scheduling.

(c) **Evaluation:** Implement the following evaluation methods:

(c1) `_calculate_perplexity`: Calculate perplexity on the test dataset.

(c2) `_calculate_repetition_score`: Measure repetition in generated text.

(c3) `_calculate_diversity_score`: Calculate lexical diversity (unique words / total words).

(c4) `_calculate_jaccard_similarity`: Calculate Jaccard similarity between true and generated text.

(d) **Fine-tuning:** After implementing the methods:

(d1) Fine-tune the GPT-2 model on the complete Alpaca test dataset.

(d2) Adjust hyperparameters (`epochs`, `batch_size`, `learning_rate`) for optimal performance.

(d3) Generate responses from the pre-trained GPT-2 and the fine-tuned model on the test dataset.

(d4) Evaluate the performance of both models on the test dataset using the implemented evaluation methods.

(e) **Analysis:** Analyze the fine-tuned model's performance compared to the pre-trained model. Include the following in your analysis:

(e1) A qualitative analysis of the generated responses.

(e2) A quantitative analysis of model's performance, including perplexity, repetition score, diversity score, and Jaccard similarity.

(e3) A brief discussion of the improvements observed after fine-tuning compared to the pre-trained model.

(f) **Submission:** Include the following in your submission:

- The analysis report summarizing the results of the fine-tuning process (one report for both tasks).
- Your fine-tuning implementation `finetuning.py`.
- Evaluation results for pre-trained and fine-tuned models on the full test dataset.
- Generated outputs from both pretrained and fine-tuned models.
- Log file `log_GROUP_XX.log`.